

CAPITOLO 14

Il metodo del Branch and Bound

14.1 Cenni generali

Il metodo del Branch and Bound permette di determinare la soluzione ottima di un problema di Programmazione Lineare a Numeri Interi (PLI).

Essendo, in generale, un PLI un problema matematicamente non trattabile, il metodo Branch and Bound conduce allo sviluppo di algoritmi esponenziali.

Branch = Ramo, Ramificazione, Separazione.

Cioè

Processo di partizionamento della regione ammissibile

Bound = Limite (superiore & inferiore)

Cioè...

Processo di stima del costo di un problema

Fase di branch

- Sia P un problema di Ottimizzazione Lineare a Numeri Interi la cui regione ammissibile è $\Omega(P)$

P si dice *separato* nei sottoproblemi $P_1, P_2, \dots, P_q$ se, indicate con $\Omega(P_i)$ le regioni ammissibili dei sottoproblemi P_i , si ha che :

- $\Omega(P_1) \cup \Omega(P_2) \cup \dots \cup \Omega(P_q) = \Omega(P)$
- $\Omega(P_i) \cap \Omega(P_j) = \emptyset \quad \forall i, j \in [1, \dots, q] \quad i \neq j$

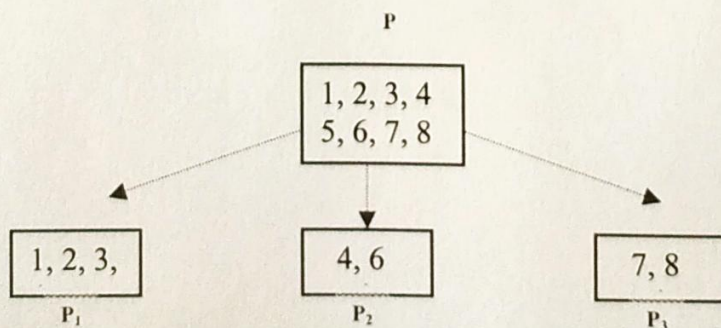
La prima condizione indica che i problemi P_i sono esaustivi, ovvero che ogni soluzione ammissibile di P è anche soluzione ammissibile di uno dei P_i e, viceversa, che ogni soluzione di un P_i è anche soluzione ammissibile di P .

La seconda condizione indica che i P_i sono mutuamente escludentesi.

- Il concetto su cui è basato il metodo è quello di decomporre un problema difficile P in sottoproblemi più semplici da risolvere P_i .

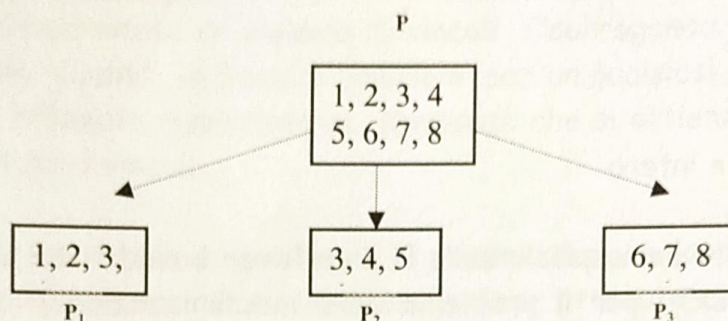
Esempio : Dato un problema la cui regione ammissibile $\Omega(P)$ si compone di 8 soluzioni ammissibili numerate da 1 a 8, si considerino le seguenti situazioni:

- Branch non corretto



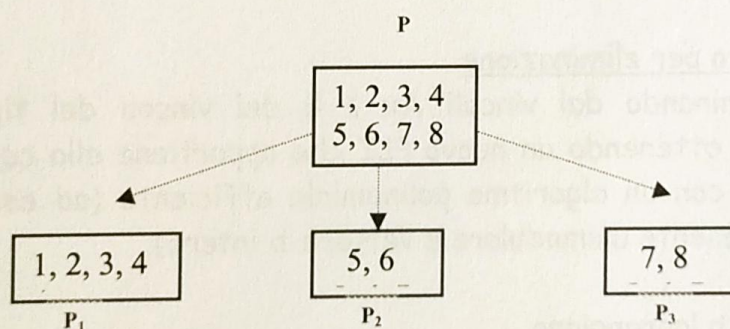
Nei sottoproblemi manca la soluzione 5. L'unione delle regioni ammissibili non dà la regione ammissibile originale.

- Branch non corretto



La soluzione 3 figura in più sottoproblemi (P_1 e P_2). Quindi le regioni ammissibili non sono disgiunte.

- Branch corretto



Adesso a ciascun sottoproblema è associata una regione ammissibile che è una parte di quella originale e non contiene soluzioni appartenenti anche agli altri sottoproblemi. Inoltre l'unione di tutte e tre le regioni ammissibili dà la regione ammissibile originale

Fase di bound

Si consideri un PLI nella forma :

$$\begin{array}{ll}\text{Min} & cx \\ & Ax \geq b \\ & x \geq 0 \text{ e intero}\end{array}$$

Come si è visto nel capitolo sulla PLI un lower bound $LB \leq cx^*$ (upper bound $UB \geq cx^*$ per il problema della massimizzazione) può essere ottenuto in vari modi :

a) Rilassamento lineare

Si ottiene eliminando i vincoli di interezza e risolvendo con il semplice (o con altro metodo) il PL ottenuto.

b) Rilassamento per eliminazione

Si ottiene eliminando dai vincoli $Ax \geq b$ dei vincoli del tipo " side constraints " e ottenendo un nuovo PLI che appartiene alla categoria di quelli risolubili con un algoritmo polinomiale efficiente (ad esempio con matrice A totalmente unimodulare e vettore b intero).

c) Rilassamento lagrangiano

Si ottiene, come quello precedente rimuovendo i "side constraints", tali vincoli però vengono anche (parzialmente o nella loro totalità) penalizzati lagrangianamente e introdotti nella funzione obiettivo.

d) Introducendo ulteriori vincoli che siano ridondanti per il politopo che definisce la regione ammissibile del PLI. Tali disuguaglianze sono dette anche "tagli " , perché esse tagliano via parte del Politopo, lasciando però tutti i punti interi ammissibili contenuti in esso,

consentendo così di arrivare a bounds più "stretti" (cioè più vicini all'ottimo).

e) Rilassamento surrogato

Combinando linearmente un insieme di vincoli ("surrogando") in un unico vincolo lineare. Quindi, in linea di principio, per un qualsiasi PLI un lower bound viene ottenuto risolvendo un *Knapsack* che si ottiene combinando linearmente tutti i vincoli.

- Un upper bound in un problema di minimizzazione (oppure un lower bound in un problema di massimizzazione) viene ottenuto considerando il costo di una qualsiasi soluzione ammissibile x (ad esempio ottenuta mediante un algoritmo euristico)

14.2 Il concetto di enumerazione implicita

• Dato un problema P di Ottimizzazione Lineare a Numeri Interi, in linea teorica è possibile enumerare tutte le possibili soluzioni in quanto il loro numero, a differenza della PL, è finito (ad esempio se n è il numero delle variabili decisionali ed N l'intero più grande, un upper bound sul numero di soluzioni è dato da $(N+1)^n$, 2^n nel caso di programmazione binaria), valutarne il costo e, quindi, scegliere quella più conveniente.

Questo tipo di enumerazione è detta esplicita e non è efficiente poiché il numero di soluzioni cresce rapidamente al crescere delle dimensioni dell'istanza che si deve risolvere.

Esempio :

$$\begin{array}{ll}
 \text{Min} & -4x_1 + 3x_2 - 6x_3 \\
 \text{P:} & 2x_1 + 3x_2 + 5x_3 \leq 11 \\
 & x_1, x_2, x_3 \in \{0, 1\}
 \end{array}$$

Enumerazione totale:

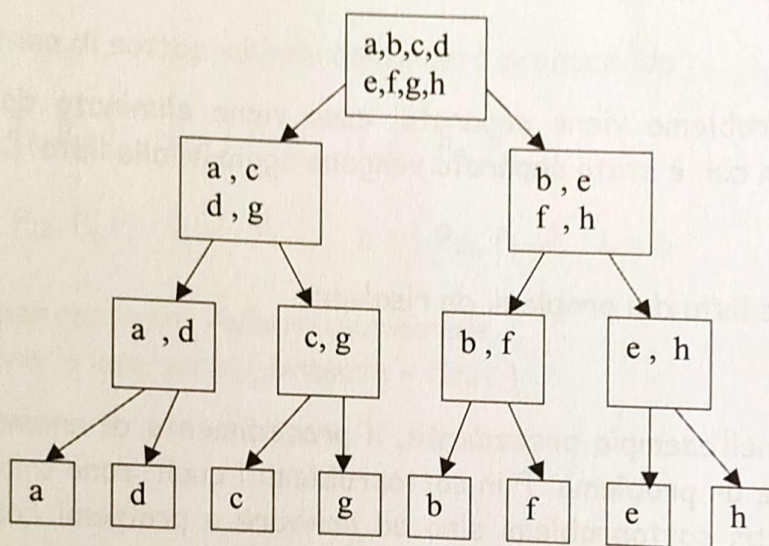
Soluzione	Costo
a) $x_1 = 0, x_2 = 0, x_3 = 0$	$z=0$
b) $x_1 = 1, x_2 = 0, x_3 = 0$	$z=-4$
c) $x_1 = 0, x_2 = 1, x_3 = 0$	$z=3$
d) $x_1 = 0, x_2 = 0, x_3 = 1$	$z=-6$
e) $x_1 = 1, x_2 = 1, x_3 = 0$	$z=-1$
f) $x_1 = 1, x_2 = 0, x_3 = 1$	$z=-10$ ottimo
g) $x_1 = 0, x_2 = 1, x_3 = 1$	$z=-3$

$$h) x_1 = 1, x_2 = 1, x_3 = 1$$

$$z = -7$$

• Tutte le soluzioni generate dall'enumerazione totale possono essere rappresentate su un albero decisionale (branching tree) in cui il nodo radice corrisponde al problema originario P.

Ogni problema relativo ad un nodo viene separato in due o più sottoproblemi fino a che non si giunga ad un sottoproblema contenente un unico punto soluzione.



- Il processo di " branching " necessario per enumerare tutte le soluzioni e costruire l'albero decisionale consiste in una regola con cui, dato un problema P , lo si separa ogni volta in q sottoproblemi (nell'esempio precedente $q = 2$).
- In tal modo viene introdotta una lista L di problemi che inizialmente contiene il solo problema P .
- Quando un problema viene separato, esso viene eliminato da L e i sottoproblemi in cui è stato separato vengono aggiunti alla lista L .
- La lista L è la lista dei problemi da risolvere.
- Come visto nell'esempio precedente, il procedimento di enumerazione totale scompone un problema P in sottoproblemi i quali sono a loro volta scomposti in altri sottoproblemi sino ad arrivare a problemi contenenti una sola soluzione (i nodi foglia d'albero decisionale).
- oltre alla tecnica del branching occorre una strategia per determinare la sequenza dei problemi da scomporre, ovvero una strategia per esplorare l'albero decisionale.

Esempio

Dato il problema P_0 , lo si inserisce nella lista $L = \{ P_0 \}$,

Si scompone P_0 in P_1 e P_2 , quindi si introducono tali problemi in $L = \{ P_1, P_2 \}$ estraendone P_0 .

Si scompone P_1 in P_3 e P_4 e si toglie P_1 da L , per cui $L = \{ P_3, P_4, P_2 \}$

Continuando nella scomposizione si ottiene : $L = \{ P_7, P_8, P_4, P_2 \} \dots$ i problemi P_7 e P_8 , contenendo una sola soluzione, si possono risolvere direttamente (in realtà basta valutare il costo dell'unica soluzione) ed eliminare da $L = \{ P_4, P_2 \}$

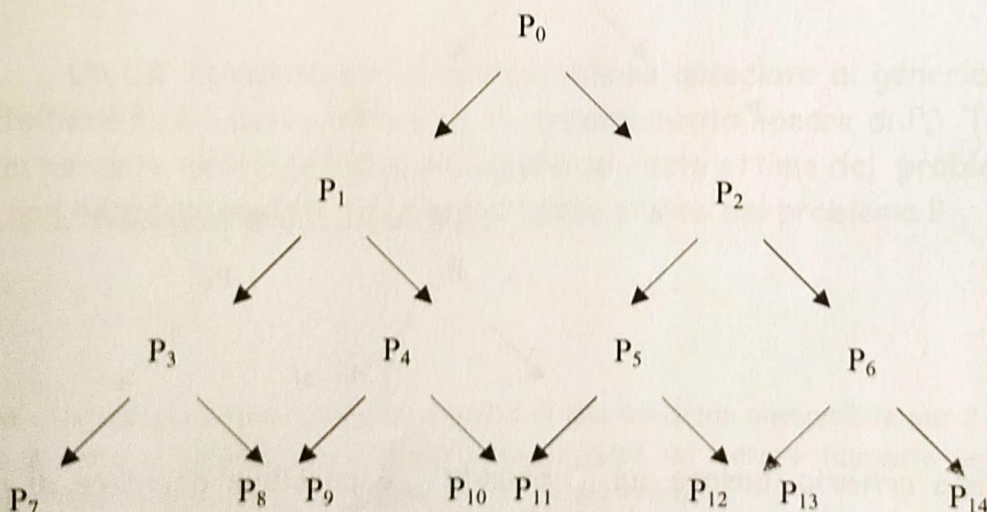
La sequenza di sottoproblemi continuerà producendo :

$L = \{ P_2, P_9, P_{10} \}$; $L = \{ P_2 \}$; $L = \{ P_5, P_6 \}$;

$L = \{ P_{11}, P_{12}, P_6 \}$; $L = \{ P_6 \}$; $L = \{ P_{13}, P_{14} \}$; $L = \phi$

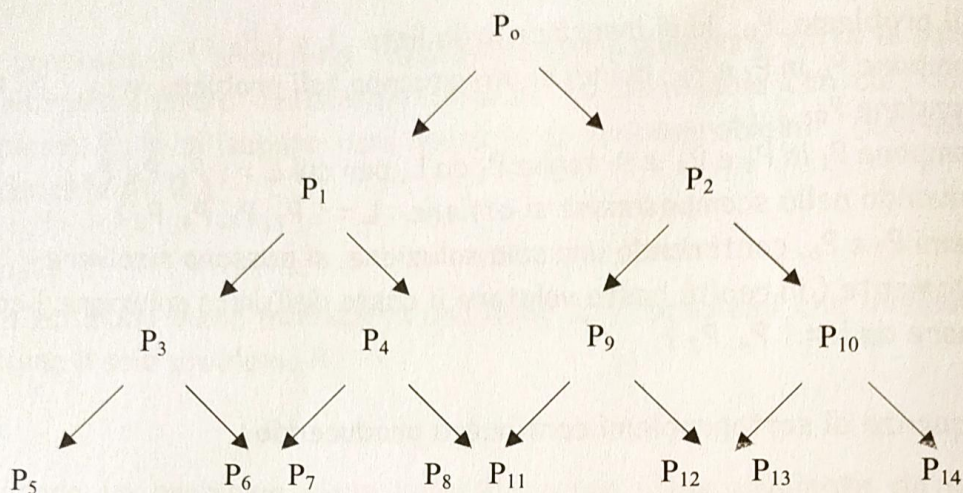
Criteri per costruire l'albero decisionale

Espansione in larghezza (breadth - first)



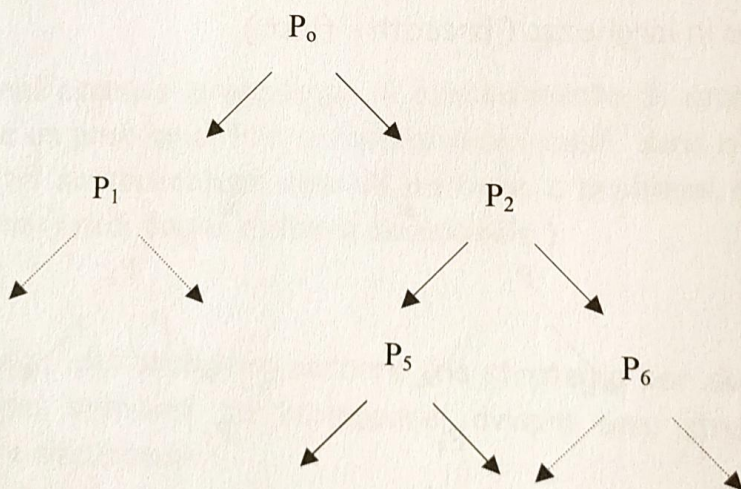
Si esaminano prima i nodi ad uno stesso livello.

Espansione in profondità (depth - first)



Espansione sul problema più promettente (best bound first)

La scelta del problema più promettente è basata sul calcolo dei "bounds",
come vedremo in seguito



Nell'ultimo criterio, basato sui " bounds", è possibile decidere di non esplorare subito alcuni problemi ritenuti non promettenti, cioè alcuni problemi per cui si ritiene che non contengano soluzioni di costo migliore rispetto ad altre soluzioni precedentemente determinate.

Rilassamento

Sono praticabili i vari tipi di rilassamento esaminati nel capitolo precedente. Esaminiamo in questo esempio il rilassamento lineare P_R di P che si ottiene semplicemente eliminando i vincoli di interezza :

$$\text{Min } -4x_1 + 3x_2 - 6x_3$$

$$P_R \quad 2x_1 + 3x_2 + 5x_3 \leq 11$$

$$x_1, x_2, x_3 \geq 0$$

Risolvendo all'ottimo P_R (ad esempio con il semplice) si ottiene

$$LB = -22$$

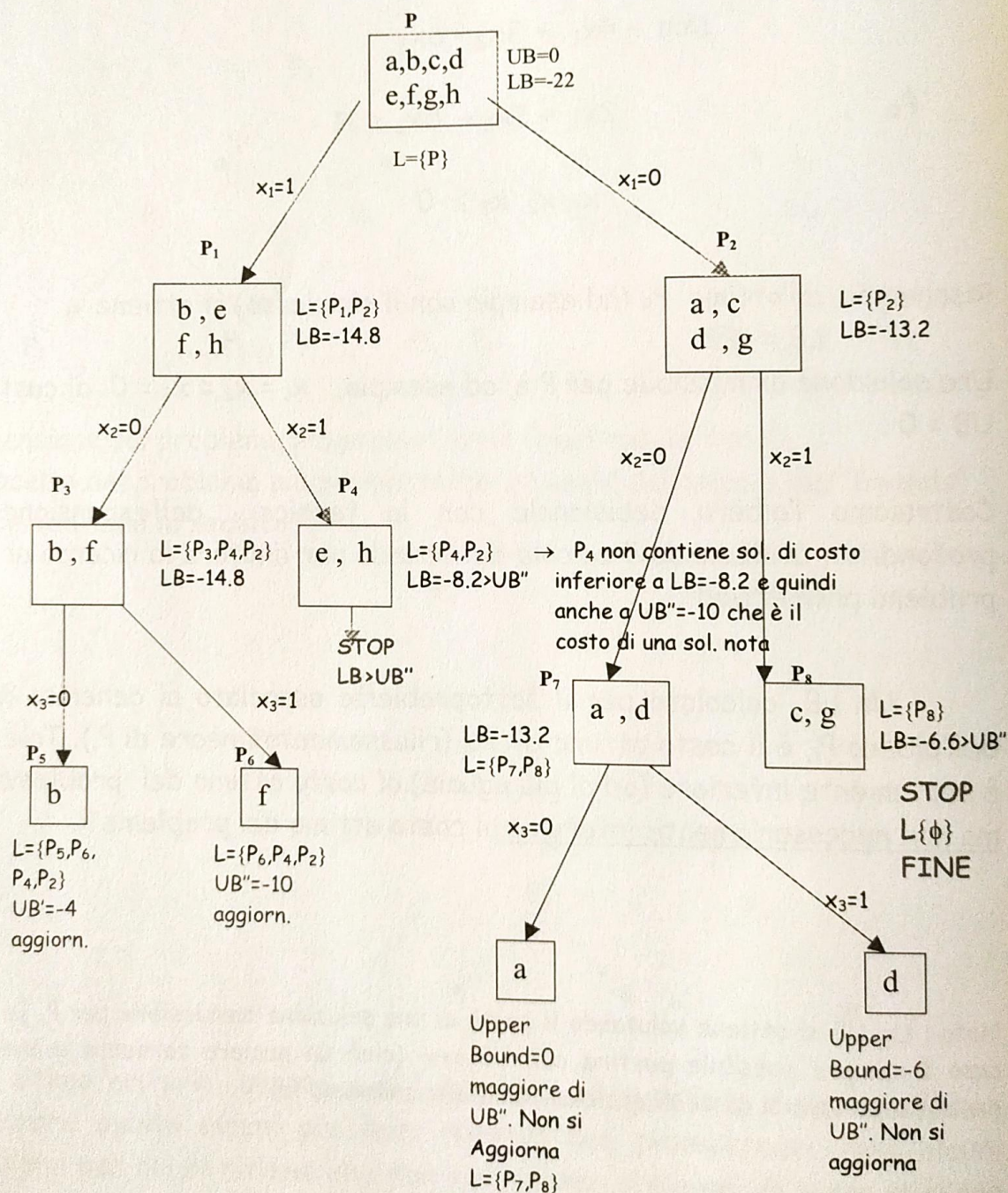
Una soluzione ammissibile per P è, ad esempio, $x_1 = x_2 = x_3 = 0$ di costo $UB = 0$.

Costruiamo l'albero decisionale con la tecnica dell'espansione in profondità, utilizzando il calcolo dei bounds per limitare la ricerca ai soli problemi promettenti.

Un LB , calcolato per il sottoproblema associato al generico nodo dell'albero P_i , è il costo ottimo di P_{Ri} (rilassamento lineare di P_i). Tale LB è certamente inferiore (od al più uguale) al costo ottimo del problema P_i , ma non necessariamente inferiore al costo ottimo del problema P .

Nota : Un UB si ottiene valutando il costo di una soluzione ammissibile per P . In ogni caso è sempre possibile partire con $UB \rightarrow +\infty$ (cioè un numero talmente grande da essere superiore al costo di qualsiasi soluzione ammissibile).

Con riferimento all'albero decisionale creato dal procedimento di enumerazione esplicita, l'albero decisionale dell'enumerazione implicita è composto da un numero minore di nodi, e non tutte le soluzioni di P sono valutate.



Il procedimento di branching operato nell'esempio è stato effettuato assegnando per ogni ramo dell'albero uno dei due valori possibili ad una delle variabili decisionali.

Nell'esempio precedente attraverso l'enumerazione implicita sono state valutate solo 4 delle 8 soluzioni .

- Utilizzando il calcolo dei bounds non tutte le soluzioni sono state generate.

Infatti, se su un nodo il lower bound è risultato superiore all'upper bound (la miglior soluzione trovata fino a quel momento) allora il nodo è stato "chiuso" perché tutte le soluzioni generabili a partire da esso, avendo costo superiore od al più uguale al lower bound e quindi all'upper bound corrente, sicuramente non sono migliori di una soluzione già determinata.

- Il procedimento di enumerazione implicita visto nel precedente esempio, che è composto di fasi di "branch" e di fasi di "bound" sarà formalizzato nella prossima sezione sotto il nome di algoritmo Branch & Bound.

Nota : I lower bound calcolati per ogni sottoproblema P_i sono lower bound locali e non globali. Un lower bound globale, cioè un lower bound sulla soluzione ottima di P , è pari al minimo dei lower bound locali dei sottoproblemi aperti (cioè i sottoproblemi presenti nella lista L).